



نام درس: شبکه های کامپیوتری
نویسنده: محمد امانلو - ۸۱۰۱۰۰۰۸۴



خلاصه شماره
۲۷

در این بخش، کل بحث روی این می‌چرخد که در لایه شبکه، «تصمیم‌گیری» از جنس Control Plane دقیقاً چه چیزهایی را پوشش می‌دهد و چرا اصلاً لازم شده نگاه‌مان را از مدل کلاسیکِ روترمحور به سمت مدل‌های جدیدتر ببریم. نقشه راهی که برای Control Plane داده می‌شود، از مدیریت و پیکربندی شبکه شروع می‌کند مثل SNMP و NETCONF/YANG، بعد می‌رود سراغ پروتکل‌های مسیریابی درون یک ISP مثل OSPF و مسیریابی بین ISP‌ها مثل BGP، سپس وارد دنیای SDN می‌شود و در نهایت هم ICMP را به عنوان زبان پیام‌های کنترلی همراه IP جمع‌بندی می‌کند.

اگر برگردیم به مدل تاریخی اینترنت، لایه شبکه سال‌ها عمدتاً با یک رویکرد «توزیع‌شده و داخل هر روتر» پیاده‌سازی شده است: هر روتر یک جعبه تقریباً یکپارچه است که هم سخت‌افزار سویچینگ دارد، هم یک سیستم عامل اختصاصی مثل Cisco IOS، و هم پیاده‌سازی اختصاصی پروتکل‌های استاندارد اینترنت را اجرا می‌کند؛ از خود IP گرفته تا پروتکل‌های کنترل مثل RIP، IS-IS، OSPF و BGP. کنار این‌ها هم برای هر کار شبکه‌ای یک middlebox جدا می‌نشست: firewall، load balancer، NAT و چیزهای مشابه. مشکل این مدل این نیست که کار نمی‌کند؛ اتفاقاً سال‌ها جواب داده، اما وقتی شبکه بزرگ‌تر و سیاست‌ها پیچیده‌تر می‌شوند، این تکه‌تکه بودن و این که «هر روتر برای خودش» تصمیم بگیرد، مدیریت و تغییر دادن رفتار شبکه را سخت و پریسک می‌کند؛ برای همین هم حوالی 2005~ دوباره بحث «بازاندیشی Control Plane» جدی می‌شود.

اینجاست که Software-Defined Networking (SDN) وارد می‌شود و ایده مرکزی‌اش را خیلی ساده می‌گوید: بگذاریم Data Plane تا جای ممکن سریع و ساده باشد و فقط «فوروارد» کند، ولی تصمیم‌های کنترلی را از داخل دستگاه‌ها بیرون بکشیم و بدهیم به یک کنترل‌کننده بیرونی. در مدل کلاسیکِ per-router control plane، اجزای الگوریتم مسیریابی در تک‌تک روترها اجرا می‌شوند و با هم تعامل می‌کنند تا هر روتر جدول فورواردینگ خودش را بسازد؛ اما در مدل SDN control plane، یک کنترلرِ راه‌دور (کنترلرِ منطقی) جدول‌ها را حساب می‌کند و روی روترها/سویچ‌ها نصب می‌کند.

نکته کلیدی این است که «مرکزیت» در SDN معمولاً «مرکزیت منطقی» است، نه لزوماً یک ماشین واحد؛ یعنی از نگاه طراحی انگار یک مغز شبکه‌سراسری داریم که همه چیز را می‌بیند، ولی در عمل برای کارایی، مقیاس‌پذیری و تحمل‌پذیری خطا، همین کنترلر به صورت یک سیستم توزیع‌شده پیاده‌سازی می‌شود. خود اسلایدها دلیل این مرکزیت منطقی را هم خیلی شفاف می‌گویند: مدیریت شبکه ساده‌تر می‌شود و احتمال بدپیکربندی روترها پایین

می آید؛ انعطاف پذیری جریان های ترافیکی بیشتر می شود؛ و چون فورواردینگ به شکل «جدول محور» انجام می شود (با یادآوری ایده های OpenFlow API) عملاً می توان روتر/سوییچ را «برنامه پذیر» کرد. برنامه پذیری اگر متمرکز باشد راحت تر است: جدول ها را مرکزی حساب می کنی و پخش می کنی؛ در حالی که برنامه پذیری توزیع شده یعنی باید خروجی یک الگوریتم توزیع شده (یک پروتکل) را در تک تک روترها پیاده کنی و هماهنگ نگه داری، که دردسرش خیلی بیشتر است. تازه چون این مدل «باز و غیرانحصاری» است، نوآوری را هم بیشتر می کند و عملاً می گذارد «هزار گل شکوفا شود» و ایده های مختلف در لایه کنترل رشد کنند.

برای جا انداختن همین تفاوت، یک قیاس هم مطرح می شود: قیاس انقلاب mainframe به PC. در مدل عمودی قدیمی، همه چیز بسته و اختصاصی است: سخت افزار اختصاصی، سیستم عامل اختصاصی، اپلیکیشن های اختصاصی؛ نوآوری کند، صنعت کوچک. در مدل افقی جدید، open interface می آیند، اجزا جدا می شوند، بازار بزرگ تر می شود و نوآوری سریع تر جلو می رود. SDN هم دقیقاً به همین سمت هل می دهد: جدا کردن نقش ها و استاندارد کردن رابط ها.

بعد، برای اینکه حس کنیم چرا با مسیریابی سنتی کارها گاهی «قفل» می شود، بحث Traffic Engineering را مطرح می کند. در مسیریابی سنتی، اپراتور شبکه اگر بخواهد ترافیک u-to-z را به جای مسیر uxyz از مسیر uvwz عبور بدهد، غالباً تنها اهرمی که واقعاً دستش هست دستکاری «وزن لینک ها» است تا الگوریتم مسیریابی خودش به آن مسیر برسد؛ یعنی به جای اینکه مستقیم بگویی «این جریان را از این مسیر ببر»، باید وزن ها را آن قدر بازی بدهی تا الگوریتم خروجی مطلوب بدهد، و تازه ممکن است ناخواسته مسیرهای دیگر را هم به هم بریزد. بدترش وقتی است که بخواهی ترافیک u-to-z را بین دو مسیر تقسیم کنی (load balancing واقعی)؛ اسلاید می گوید با مسیریابی کلاسیک عملاً نمی شود (یا باید الگوریتم جدید بیاوری). حتی اگر یک گره مثل w بخواهد ترافیک blue و red را از w به z متفاوت مسیر بدهد، با destination-based forwarding و مسیریابی های LS و DV عملاً این تفکیک طبیعی نیست. جمع بندی این بخش هم این است که در فصل قبل یاد گرفتیم «فورواردینگ تعمیم یافته» و SDN می توانند هر نوع مسیریابی دلخواهی را عملی کنند، چون دیگر دستمان فقط به مقصد بسته نیست و می توانیم روی «جریان ها» سیاست بگذاریم.

حالا وقتی وارد معماری SDN می شویم، تصویر کلی سه تکه است: سویچ های Data Plane، کنترلر SDN (یا همان network OS)، و اپلیکیشن های کنترلی. سویچ های دیتا پلن قرار است «سریع، ساده و commodity» باشند و در سخت افزار، فورواردینگ جدول محور تعمیم یافته را اجرا کنند؛ یک flow (forwarding) table دارند که تحت نظارت کنترلر محاسبه و نصب می شود. برای اینکه معلوم باشد چه چیزهایی قابل کنترل است و چه چیزهایی نه، یک API برای کنترل جدول ها داریم (مثلاً OpenFlow) و همچنین یک پروتکل برای ارتباط با کنترلر (باز هم مثلاً OpenFlow). کنترلر SDN نقش نگه داری «وضعیت شبکه» را دارد و با اپلیکیشن های کنترلی در بالا از طریق northbound API حرف می زند و با سویچ ها/روترها در پایین از طریق southbound API. خود کنترلر معمولاً به صورت یک سیستم

توزیع شده پیاده می شود تا هم performance داشته باشد، هم scalability، هم fault-tolerance و هم robustness و در نهایت اپلیکیشن های کنترلی همان «مغز سیاست ها» هستند: چیزهایی مثل routing، access control و load balance. نکته جذاب این معماری این است که اپلیکیشن ها unbundled هستند؛ یعنی می توانند توسط 3rd party هم ساخته شوند و لزوماً وابسته به فروشنده روتر یا حتی سازنده کنترلر نیستند.

داخل خود کنترلر هم سه مؤلفه اصلی برجسته می شود: یک بخش communication برای ارتباط بین کنترلر و سویچ های تحت کنترل، یک بخش network-wide state management که وضعیت لینک ها، سویچ ها و سرویس ها را به صورت یک «پایگاه داده توزیع شده» نگه می دارد، و یک interface layer که به اپلیکیشن های کنترلی انتزاع و API می دهد تا لازم نباشد همه چیز را در سطح پیام های خام و جزئیات ریز بنویسند.

برای ارتباط پایین دست، OpenFlow protocol معرفی می شود که بین کنترلر و سویچ کار می کند و معمولاً روی TCP پیام رد و بدل می کند و می تواند optional encryption هم داشته باشد. یک نکته مهم این است که Open-Flow protocol با OpenFlow API یکی نیست: پروتکل، «قالب پیام ها و تعامل» را تعیین می کند، ولی API می گوید «چه اعمال فورواردینگی» را می شود مشخص کرد تا فورواردینگ تعمیم یافته شکل بگیرد. از نظر دسته بندی پیام ها، سه گروه کلی داریم: پیام های controller-to-switch، پیام های asynchronous از switch-to-controller و پیام های symmetric که متفرقه اند. در پیام های کنترلر به سویچ، پیام های کلیدی این ها هستند: کنترلر می تواند features سویچ را بپرسد و پاسخ بگیرد، تنظیمات را configure کند، وضعیت جدول را با modify-state تغییر بدهد (اضافه/حذف/ویرایش flow entry ها)، و حتی یک بسته را با packet-out از یک پورت مشخص بیرون بفرستد. در سمت دیگر، سویچ با packet-in بسته ای را که نیاز به تصمیم کنترلی دارد (یا با جدول فعلی جور در نمی آید) به کنترلر می فرستد، با flow-removed حذف شدن یک ورودی جدول را گزارش می دهد، و با port status تغییر وضعیت پورت ها را اطلاع می دهد. و یک واقعیت عملی هم گفته می شود: اپراتورهای شبکه معمولاً مستقیم «پیام های OpenFlow» را دست نویس نمی کنند؛ در عمل از انتزاع های سطح بالاتر خود کنترلر استفاده می کنند تا با جزئیات پیام ها درگیر نشوند.

برای اینکه این تعامل Control/Data Plane ملموس شود، یک مثال از این می آید که کنترلر می تواند منطق مسیریابی Dijkstra's link-state routing را اجرا کند و خروجی اش را به شکل جدول های فورواردینگ روی تجهیزات نصب کند؛ یعنی همان کاری که قبلاً هر روتر به تنهایی و با تبادل پیام های پروتکلی انجام می داد، حالا با دید شبکه سراسری و به شکل متمرکز/منطقی انجام می شود و بعد روی دیتاپلن «اعمال» می گردد.

بعد از معماری و OpenFlow، دو نمونه کنترلر معروف هم معرفی می شوند: OpenDaylight (ODL) و ONOS. در ODL یک لایه مهم به نام Service Abstraction Layer (SAL) دیده می شود که مثل یک چسب، اپلیکیشن ها و سرویس های داخلی و خارجی را به هم وصل می کند و در کنار Northbound API و Southbound API، نقش «یک دست سازی» ارتباطات را دارد. در ONOS هم چند تأکید پررنگ وجود دارد: اپلیکیشن های کنترلی از خود کنترلر

جدا هستند؛ intent framework کمک می کند سرویس را در سطح «چه می خواهیم» تعریف کنی نه «چطور انجام بده»؛ هسته کنترلر یک distributed core جدی دارد که روی performance, replication, service reliability و scaling حساس است؛ و بحث hardening the control plane مطرح می شود، یعنی کنترلر پلن باید از روز اول مثل یک سیستم توزیع شده قابل اتکا و امن طراحی شود تا در برابر خرابی ها مقاوم باشد و امنیتش baked in باشد. این نگاه برای شبکه هایی با نیازهای مأموریت محور مثل ultra-reliable, real-time و ultra-secure مهم تر هم می شود، و حتی اشاره می شود که SDN در شبکه های سلولی 5G نقش حیاتی پیدا کرده است، و همچنین وقتی از مقیاس اینترنت حرف می زنیم باید فراتر از یک AS هم فکر کنیم.

در ادامه، یک نگاه آینده محور هم داده می شود که تفاوت «جدول های فورواردینگ محاسبه شده توسط SDN» با «جدول های محاسبه شده توسط روترها با پروتکل ها» فقط یک نمونه است از یک ایده بزرگ تر: خیلی از کارهایی که امروز با پروتکل های کاملاً توزیع شده انجام می دهیم را می شود به سمت تصمیم گیری منطقی-متمرکز برد. حتی یک تصور مطرح می شود که شاید روزی congestion control هم SDN-computed شود؛ یعنی کنترلر بر اساس گزارش ازدحام از روترها، نرخ ارسال فرستنده ها را تنظیم کند. پیام کلی این است که SDN فقط جایگزینی یک پروتکل با پروتکل دیگر نیست؛ بیشتر یک تغییر سبک طراحی است که می گوید «کجا تصمیم بگیریم» و «چطور اعمال کنیم». در بخش پایانی، ICMP مطرح می شود تا یادمان نرود کنار همه این معماری ها، اینترنت هنوز به یک زبان ساده و پایه برای «گزارش خطا و وضعیت» نیاز دارد. (Internet Control Message Protocol (ICMP) توسط میزبان ها و روترها استفاده می شود تا اطلاعات سطح شبکه را منتقل کند؛ مهم ترین کاربردهایش گزارش خطاست مثل unreachable host, unreachable network, unreachable port یا unreachable protocol. علاوه بر خطا، echo request/reply هم دارد که همان ابزار ping عملاً روی آن سوار است. از نظر لایه ای، ICMP به نوعی «بالای IP» حساب می شود، ولی پیام هایش داخل IP datagram حمل می شوند. ساختار پیام ICMP هم معمولاً شامل type و code است و برای اینکه فرستنده بفهمد مشکل دقیقاً مربوط به کدام بسته بوده، معمولاً «۸ بیت اول» دیتاگرام IP که خطا را ایجاد کرده هم در پیام می آید. در جدول نوع/کدها، موارد معروف مثل echo reply/request، انواع destination unreachable، پیام TTL expired و bad IP header دیده می شود؛ یک مورد تاریخی مثل source quench هم ذکر می شود که برای کنترل ازدحام بوده ولی عملاً not used است.

و نهایتاً traceroute توضیح داده می شود که چطور با بازی هوشمندانه با TTL و همین پیام های ICMP مسیر را کشف می کند. ایده این است که مبدا چند سری سگمنت UDP به مقصد می فرستد: سری اول با TTL=1، سری بعد با TTL=2 و همین طور افزایش می دهد. هر دیتاگرامی که به روتر

ام^۱ برسد و TTLش همان جا تمام شود، روتر آن را دور می اندازد و به مبدا یک پیام ICMP از نوع type 11, code 0 برمی گرداند که همان TTL expired است؛ مبدا با رسیدن این پیام ها RTT ها را اندازه می گیرد و گام به گام می فهمد بسته ها از کدام روترها عبور می کنند (گاهی حتی نام و IP address روتر هم در پیام دیده می شود). شرط توقف هم این است که بالاخره یکی از سگمنت های UDP به مقصد برسد و چون پورت معمولاً طوری انتخاب می شود که سرویسی

روی آن گوش نمی دهد، مقصد یک ICMP port unreachable برمی گرداند (type 3, code 3) و مبدا متوجه می شود به مقصد رسیده و تمام می کند.

مراجع